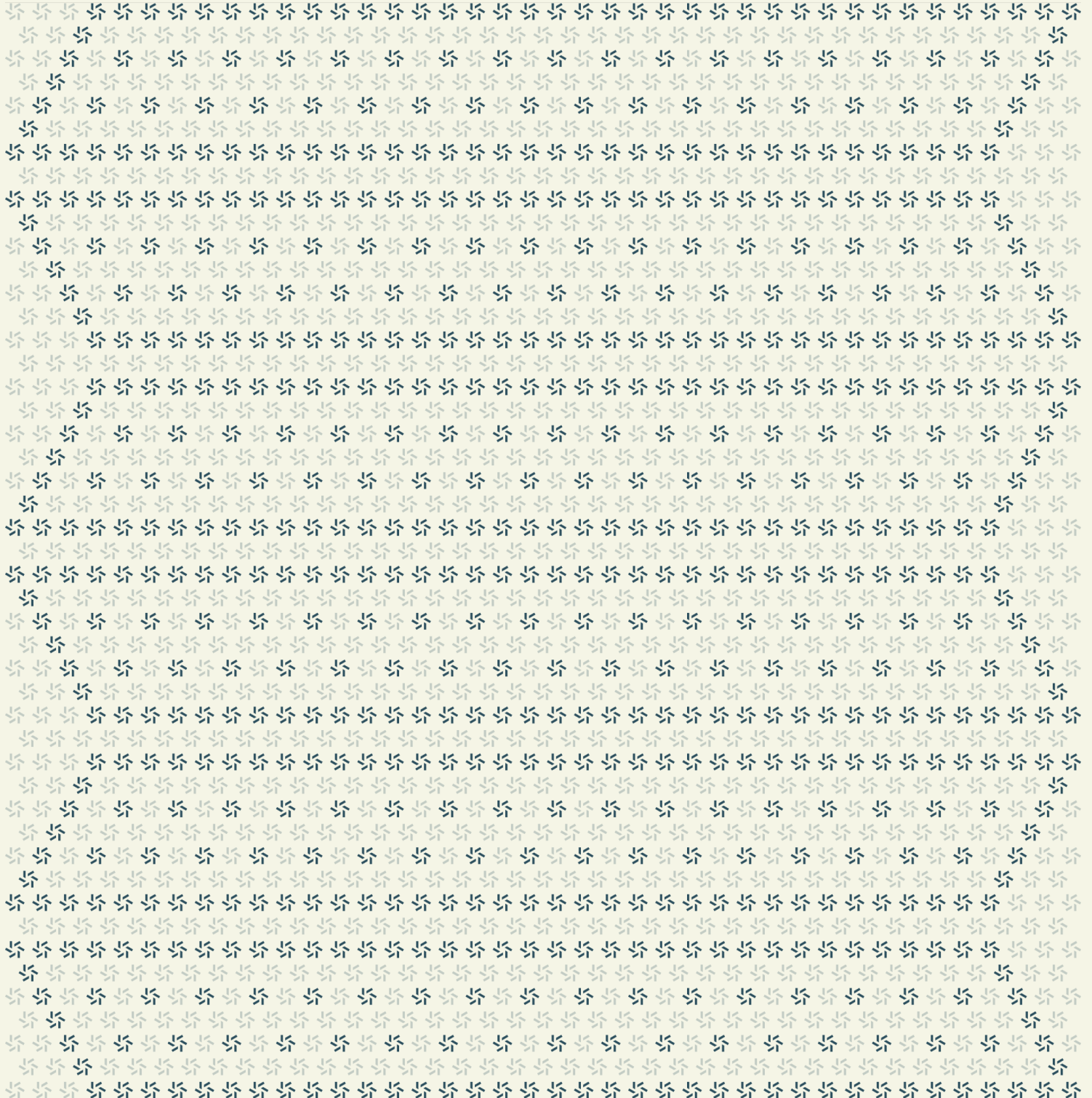


June 17, 2026

Program Metadata

Solana Application Security Assessment



Contents

About Zellic	4
1. Overview	5
1.1. Executive Summary	5
1.2. Goals of the Assessment	5
1.3. Non-goals and Limitations	5
1.4. Results	5
2. Introduction	7
2.1. About Program Metadata	7
2.2. Methodology	7
2.3. Scope	9
2.4. Project Overview	9
2.5. Project Timeline	10
3. Detailed Findings	11
3.1. Missing rent exemption check in set_data	11
3.2. Undefined behavior on pointer cast	13
4. Discussion	15
4.1. Duplicate signer checks	15
4.2. Gated logging macro	15
4.3. Struct alignment assert	16

5.	Threat Model	17
5.1.	Instruction: write	17
5.2.	Instruction: initialize	18
5.3.	Instruction: set_authority	19
5.4.	Instruction: set_data	21
5.5.	Instruction: set_immutable	22
5.6.	Instruction: trim	23
5.7.	Instruction: close	24
5.8.	Instruction: allocate	26
5.9.	Instruction: extend	27

6.	Assessment Results	29
6.1.	Disclaimer	29

About Zellic

Zellic is a vulnerability research firm with deep expertise in blockchain security. We specialize in EVM, Move (Aptos and Sui), and Solana as well as Cairo, NEAR, and Cosmos. We review L1s and L2s, cross-chain protocols, wallets and applied cryptography, zero-knowledge circuits, web applications, and more.

Prior to Zellic, we founded the [#1 CTF \(competitive hacking\) team](#) worldwide in 2020, 2021, and 2023. Our engineers bring a rich set of skills and backgrounds, including cryptography, web security, mobile security, low-level exploitation, and finance. Our background in traditional information security and competitive hacking has enabled us to consistently discover hidden vulnerabilities and develop novel security research, earning us the reputation as the go-to security firm for teams whose rate of innovation outpaces the existing security landscape.

For more on Zellic's ongoing security research initiatives, check out our website zellic.io and follow [@zellic_io](#) on Twitter. If you are interested in partnering with Zellic, contact us at hello@zellic.io.



1. Overview

1.1. Executive Summary

Zellic conducted a security assessment for Anza from May 21 to June 2nd, 2026. During this engagement, Zellic reviewed Program Metadata's code for security vulnerabilities, design issues, and general weaknesses in security posture.

1.2. Goals of the Assessment

In a security assessment, goals are framed in terms of questions that we wish to answer. These questions are agreed upon through close communication between Zellic and the client. In this assessment, we sought to answer the following questions:

- Is the business logic in the metadata program sound?
 - Are there any coding mistakes in the metadata program?
-

1.3. Non-goals and Limitations

We did not assess the following areas that were outside the scope of this engagement:

- Front-end components
- Infrastructure relating to the project
- Key custody

Due to the time-boxed nature of security assessments in general, there are limitations in the coverage an assessment can provide.

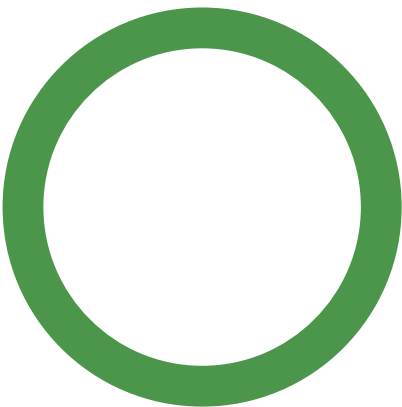
1.4. Results

During our assessment on the scoped Program Metadata programs, we discovered two findings, both of which were low impact.

Additionally, Zellic recorded its notes and observations from the assessment for the benefit of Anza in the Discussion section ([4. 7](#)).

Breakdown of Finding Impacts

Impact Level	Count
Critical	0
High	0
Medium	0
Low	2
Informational	0



2. Introduction

2.1. About Program Metadata

Anza contributed the following description of Program Metadata:

The Program Metadata provides the ability to attach metadata information to any program. The information is represented by a PDA account with a pre-defined derivation, e.g., it can be used to add the IDL of a program, with a PDA derived using the "idl" string.

2.2. Methodology

During a security assessment, Zellic works through standard phases of security auditing, including both automated testing and manual review. These processes can vary significantly per engagement, but the majority of the time is spent on a thorough manual review of the entire scope.

Alongside a variety of tools and analyzers used on an as-needed basis, Zellic focuses primarily on the following classes of security and reliability issues:

Basic coding mistakes. Many critical vulnerabilities in the past have been caused by simple, surface-level mistakes that could have easily been caught ahead of time by code review. Depending on the engagement, we may also employ sophisticated analyzers such as model checkers, theorem provers, fuzzers, and so on as necessary. We also perform a cursory review of the code to familiarize ourselves with the programs.

Business logic errors. Business logic is the heart of any smart contract application. We examine the specifications and designs for inconsistencies, flaws, and weaknesses that create opportunities for abuse. For example, these include problems like unrealistic tokenomics or dangerous arbitrage opportunities. To the best of our abilities, time permitting, we also review the contract logic to ensure that the code implements the expected functionality as specified in the platform's design documents.

Integration risks. Several well-known exploits have not been the result of any bug within the contract itself; rather, they are an unintended consequence of the contract's interaction with the broader DeFi ecosystem. Time permitting, we review external interactions and summarize the associated risks: for example, flash loan attacks, oracle price manipulation, MEV/sandwich attacks, and so on.

Code maturity. We look for potential improvements in the codebase in general. We look for violations of industry best practices and guidelines and code quality standards. We also provide suggestions for possible optimizations, such as gas optimization, upgradability weaknesses, centralization risks, and so on.

For each finding, Zellic assigns it an impact rating based on its severity and likelihood. There is no hard-and-fast formula for calculating a finding's impact. Instead, we assign it on a case-by-case basis based on our judgment and experience. Both the severity and likelihood of an issue affect

its impact. For instance, a highly severe issue's impact may be attenuated by a low likelihood. We assign the following impact ratings (ordered by importance): Critical, High, Medium, Low, and Informational.

Zellic organizes its reports such that the most important findings come first in the document, rather than being strictly ordered on impact alone. Thus, we may sometimes emphasize an "Informational" finding higher than a "Low" finding. The key distinction is that although certain findings may have the same impact rating, their *importance* may differ. This varies based on various soft factors, like our clients' threat models, their business needs, and so on. We aim to provide useful and actionable advice to our partners considering their long-term goals, rather than a simple list of security issues at present.

Finally, Zellic provides a list of miscellaneous observations that do not have security impact or are not directly related to the scoped programs itself. These observations — found in the Discussion ([4.7](#)) section of the document — may include suggestions for improving the codebase, or general recommendations, but do not necessarily convey that we suggest a code change.

2.3. Scope

The engagement involved a review of the following targets:

Program Metadata Programs

Type	Rust
Platform	SVM
Target	program-metadata
Repository	https://github.com/solana-program/program-metadata/ ↗
Version	9f4dfbacce875f4d2c2940787b8ec75bcaa96446
Programs	program/src/**/*.rs

2.4. Project Overview

Zellic was contracted to perform a security assessment for a total of 1.5 person-weeks. The assessment was conducted by two consultants over the course of 1.5 calendar weeks. This engagement is documented in two separate reports. This report covers Component 2 of the May 21st, 2026 engagement and presents the associated findings and observations. Component 1 is addressed in a separate report.

Contact Information

The following project manager was associated with the engagement:

Jacob Goreski
↗ Engagement Manager
jacob@zellic.io ↗

The following consultants were engaged to conduct the assessment:

Nathanial Lattimer
↗ Engineer
d0nut@zellic.io ↗

Maik Robert
↗ Engineer
maik@zellic.io ↗

2.5. Project Timeline

The key dates of the engagement are detailed below.

May 21, 2026 Start of primary review period

June 2, 2026 End of primary review period

3. Detailed Findings

3.1. Missing rent exemption check in set_data

Target	Program Metadata		
Category	Coding Mistakes	Severity	Low
Likelihood	Low	Impact	Low

Description

There are multiple instructions that are able to resize accounts extend, set_data, trim and write. extend, trim and write check that the account that will be resized has enough lamports for rent exemption. This is done using a check like this

```
let minimum_balance = Rent::get()?.try_minimum_balance(required_length)?;

if target_buffer.lamports() < minimum_balance {
    return Err(ProgramError::AccountNotRentExempt);
}
```

A similar check is missing from the set_data instruction where

```
unsafe { metadata.resize_unchecked(Header::LEN + data.len())? };
```

is called without verifying the minimum balance.

Impact

To keep the implementation consistent across the program, it was discussed if the missing check should be added or the existing ones be removed from the instructions. While the solana runtime ensures that accounts are rent exempt after a transaction has been processed, there are edge-cases that can lead to unintended behavior. If the check is missing, it is technically possible to write to an account that has a 0 lamport balance. The account will then be deleted after the transaction, this could lead to issues with "immutable" metadata accounts, which should not be able to be deleted.

Recommendations

Add the missing rent exemption check in the set_data instruction.

Remediation

This issue has been acknowledged by Anza, and a fix was implemented in [PR #79](#). They intend to merge these changes into the production branch.

3.2. Undefined behavior on pointer cast

Target	src/processor/initialize.rs		
Category	Soundness	Severity	Low
Likelihood	Low	Impact	Low

Description

A handful of structs in the implementation are cast from bytes into their respective types. This being a dangerous operation requires significant care in defining the types and handling the cast or transmutation operation to ensure undefined behavior does not sneak in. Unfortunately, for a few of the structs, appropriate annotations which would have ensured that the struct had a predictable memory layout were missing introducing undefined behavior.

In one example, the `Initialize` struct is defined as:

```
struct Initialize {
    pub seed: [u8; 16],
    pub encoding: u8,
    pub compression: u8,
    pub format: u8,
    pub data_source: u8,
}
```

With the dangerous cast implemented as:

```
impl Initialize {
    const LEN: usize = size_of::();

    pub(crate) unsafe fn load_unchecked(bytes: &[u8]) -> &Self {
        &*(bytes.as_ptr() as *const Self)
    }
}
```

The `Initialize` struct by default uses the "Rust Representation" which does not guarantee much about the layout except a minimum alignment and disjointedness of the inner fields. This means that even between two structs with identical declaration, instances between those struct are not guaranteed to map to each other correctly.

Impact

Without an appropriate memory layout, casting raw bytes can introduce undefined behavior.

Recommendations

We recommend annotating this struct, and the other applicable structs, with `#[repr(C)]` or `#[repr(transparent)]` (where appropriate).

Remediation

This issue has been acknowledged by Anza, and a fix was implemented in [PR #69](#). They intend to merge these changes into the production branch.

4. Discussion

The purpose of this section is to document miscellaneous observations that we made during the assessment. These discussion notes are not necessarily security related and do not convey that we are suggesting a code change.

4.1. Duplicate signer checks

Two of the instructions, `close` and `set_authority` have duplicate authority signer checks. In `close`, the authority is explicitly checked first

```
if !authority.is_signer() {
    return Err(ProgramError::MissingRequiredSignature);
}
```

and later again by calling the `validate_authority()` function, which also contains the same check again

```
#[inline(always)]
fn validate_authority<T: Account>(<
    account: &T,
    authority: &AccountView,
    program: &AccountView,
    program_data: &AccountView,
> -> Result<(), ProgramError> {
    // Authority checks.
    if !authority.is_signer() {
        return Err(ProgramError::MissingRequiredSignature);
    }
    ...
}
```

The same pattern can be found in the `set_authority` instruction, where the authority is checked explicitly first and then again in all logical branches using the `validate_authority()` function.

4.2. Gated logging macro

It was observed that the following pattern was used throughout the Metadata program with a feature gated import.

```
#[cfg(feature = "logging")]
use solana_program_log::log;
```

```
...

#[cfg(feature = "logging")]
log("my message!");
```

We suggested creating a custom macro to simplify this pattern.

```
#[macro_export]
macro_rules! gated_logging {
    ($msg:literal) => {
        #[cfg(feature = "logging")]
        solana_program_log::log($msg);
    };
}
```

With this macro, feature gated logging becomes this simple.

```
gated_logging!("my message");
```

4.3. Struct alignment assert

We recommended the use of static compile-time assertions to enforce appropriate sizing of two structs which were required to be identical sizing.

```
const _ : () = {
    assert!(std::mem::size_of::<Buffer>() == std::mem::size_of::<Header>(),
        "Header and Buffer should be the same size!");
};
```


5. Threat Model

As time permitted, we analyzed each instruction in the program and created a written threat model for the most critical instructions. A threat model documents the high-level functionality of a given instruction, the inputs it receives, and the accounts it operates on as well as the main checks performed on them; it gives an overview of the attack surface of the programs and of the level of control an attacker has over the inputs of critical instructions.

Not all instructions in the audit scope may have been modeled. The absence of a threat model in this section does not necessarily suggest that an instruction is safe.

5.1. Instruction: write

This instruction writes data to an initialized buffer account. Data to write can come from an optionally provided source buffer account, or passed as optional instruction data.

Input parameters

```
struct Write {
    offset: [u8; 4],
    data: [u8], // Optional bytes to write
}
```

Accounts

- **target_buffer**: buffer to write to.
 - Signer: No.
 - Init: No.
 - PDA: No.
 - Mutable: Yes.
- **authority**: Signer of the transaction.
 - Signer: Yes.
 - Init: No.
 - PDA: No.
 - Mutable: Yes.
- **source_buffer**: Optional buffer to read data from.
 - Signer: No.
 - Init: No.
 - PDA: No.
 - Mutable: No.

Additional checks and behavior

- correct amount of accounts must be passed
- authority must be a signer
- target_buffer must be initialized
- source_buffer must be owned by metadata program
- target_buffer must be rent exempt for the post write data size

5.2. Instruction: initialize

This instruction initializes a new metadata account for a program. It can either be a new, pre-funded account or a buffer account that has been properly allocated using the `allocate` instruction. When not using a buffer, the data to be written must be passed via the instruction data.

Input parameters

```
struct Initialize {  
    pub seed: [u8; 16],  
    pub encoding: u8,  
    pub compression: u8,  
    pub format: u8,  
    pub data_source: u8,  
}
```

Accounts

- **metadata_account**: The metadata account to be initialized.
 - Signer: No.
 - Init: No.
 - PDA: No.
 - Mutable: Yes.
- **authority**: Signer of the transaction.
 - Signer: Yes.
 - Init: No.
 - PDA: No.
 - Mutable: Yes.
- **program_account**: Program account the metadata account will belong to.
 - Signer: No.
 - Init: No.

- PDA: No.
- Mutable: No.
- **program_data_account**: Optional program data account for the program account.
 - Signer: No.
 - Init: No.
 - PDA: No.
 - Mutable: No.
- **system_program**: Optional system program.
 - Signer: No.
 - Init: No.
 - PDA: No.
 - Mutable: No.

Additional checks and behavior

- correct amount of accounts must be passed
- instruction data must be expected length
- `authority` must be a signer and must either be the program upgrade authority (for canonical metadata accounts) or be included in the seeds used to derive the `metadata_account` address (for non-canonical metadata accounts)
- `metadata_account`:
 - must be a PDA derived from the program ID and the seed
 - must not already be initialized, i.e. has to be empty or a pre-allocated buffer
 - must be rent exempt for the post write data size

CPI

- Calls the system program's `Allocate` and `Assign` instructions to initialize accounts

5.3. Instruction: `set_authority`

This instruction will set or remove the authority for a metadata or buffer account.

Input parameters

```
struct SetAuthority {
    option: [u8; 4],
    data: [u8],
```

```
}
```

Accounts

- **account:** Metadata or buffer account to set the authority for.
 - Signer: No.
 - Init: No.
 - PDA: No.
 - Mutable: Yes.
- **current_authority:** Signer of the transaction and current authority.
 - Signer: Yes.
 - Init: No.
 - PDA: No.
 - Mutable: Yes.
- **program_account:** Optional program account.
 - Signer: No.
 - Init: No.
 - PDA: No.
 - Mutable: No.
- **program_data_account:** Optional program data account.
 - Signer: No.
 - Init: No.
 - PDA: No.
 - Mutable: No.

Additional checks and behavior

- correct amount of accounts must be passed
- authority must be a signer
- account must be metadata or buffer account
 - if metadata account, must not be immutable
 - if metadata account, cannot be a non-canonical metadata account
 - must have a valid authority

5.4. Instruction: set_data

This instruction will set data to a program metadata account. The data to be written can be supplied via instruction data or via a buffer account. When setting the data to a canonical metadata account with the program upgrade authority, both program and program data accounts are required.

Input parameters

```
struct SetData<'a> {  
    encoding: u8,  
    compression: u8,  
    format: u8,  
    data_source: u8,  
    bytes_to_write: [u8], // Optional bytes to write  
}
```

Accounts

- **metadata_account**: Metadata account where the data will be written to.
 - Signer: No.
 - Init: No.
 - PDA: No.
 - Mutable: Yes.
- **authority**: Signer of the transaction and current authority.
 - Signer: Yes.
 - Init: No.
 - PDA: No.
 - Mutable: Yes.
- **buffer_account**: Optional buffer account to read data from.
 - Signer: No.
 - Init: No.
 - PDA: No.
 - Mutable: No.
- **program_account**: Optional program account.
 - Signer: No.
 - Init: No.
 - PDA: No.

- Mutable: No.
- **program_data_account**: Optional program data account.
 - Signer: No.
 - Init: No.
 - PDA: No.
 - Mutable: No.

Additional checks and behavior

- correct amount of accounts must be passed
- authority must be a signer
- authority must match the authority set on the metadata_account or it must be the program upgrade authority if the metadata_account is canonical
- metadata_account must be initialized
- metadata_account must not be immutable

5.5. Instruction: set_immutable

This instruction will make an initialized metadata account immutable, preventing any further modifications to the account.

Input parameters

```
// No input parameters
```

Accounts

- **metadata_account**: Metadata to set the immutable flag for.
 - Signer: No.
 - Init: No.
 - PDA: No.
 - Mutable: Yes.
- **authority**: Signer of the transaction and current authority.
 - Signer: Yes.
 - Init: No.
 - PDA: No.
 - Mutable: Yes.

- **program_account**: Optional program account.
 - Signer: No.
 - Init: No.
 - PDA: No.
 - Mutable: No.
- **program_data_account**: Optional program data account.
 - Signer: No.
 - Init: No.
 - PDA: No.
 - Mutable: No.

Additional checks and behavior

- correct amount of accounts must be passed
- authority must be a signer
- authority must match the authority set on the `metadata_account` account or it must be the program upgrade authority if the `metadata_account` account is canonical
- `metadata_account` must be initialized
- `metadata_account` must not be immutable

5.6. Instruction: trim

This instruction can resize a buffer or metadata account and will withdraw excess lamports not required for rent exemption.

Input parameters

```
// No input parameters
```

Accounts

- **account**: Buffer or metadata account to resize or withdraw excess lamports from.
 - Signer: No.
 - Init: No.
 - PDA: No.
 - Mutable: Yes.
- **authority**: Signer of the transaction and current authority.

- Signer: Yes.
- Init: No.
- PDA: No.
- Mutable: Yes.
- **program_account**: Optional program account.
 - Signer: No.
 - Init: No.
 - PDA: No.
 - Mutable: No.
- **program_data_account**: Optional program data account.
 - Signer: No.
 - Init: No.
 - PDA: No.
 - Mutable: No.
- **destination_account**: Destination account that will receive the excess lamports.
 - Signer: No.
 - Init: No.
 - PDA: No.
 - Mutable: Yes.
- **rent_sysvar**: Rent sysvar account.
 - Signer: No.
 - Init: No.
 - PDA: No.
 - Mutable: No.

Additional checks and behavior

- correct amount of accounts must be passed
- Program ownership and writable checks are done implicitly by the runtime
- authority must match the authority set on the account account
- account must be a buffer of metadata account
- account must have a valid authority

5.7. Instruction: close

This instruction will close a program-owned buffer or metadata account.

Input parameters

```
// No input parameters
```

Accounts

- **account:** Buffer or metadata account to close.
 - Signer: No.
 - Init: No.
 - PDA: No.
 - Mutable: Yes.
- **authority:** Signer of the transaction and current authority.
 - Signer: Yes.
 - Init: No.
 - PDA: No.
 - Mutable: Yes.
- **program_account:** Optional program account.
 - Signer: No.
 - Init: No.
 - PDA: No.
 - Mutable: No.
- **program_data_account:** Optional program data account.
 - Signer: No.
 - Init: No.
 - PDA: No.
 - Mutable: No.
- **destination_account:** Destination account that will receive the excess lamports.
 - Signer: No.
 - Init: No.
 - PDA: No.
 - Mutable: Yes.

Additional checks and behavior

- correct amount of accounts must be passed
- account must have data
- authority must match the authority set on the account account

5.8. Instruction: allocate

This instruction will allocate a new buffer account. The buffer account can then be used by other instructions of this program.

Input parameters

```
struct Allocate {  
    seed: [u8; 16], // Optional seed  
}
```

Accounts

- **buffer:** Buffer account to allocate.
 - Signer: No.
 - Init: No.
 - PDA: No.
 - Mutable: Yes.
- **authority:** Signer of the transaction and authority for the buffer.
 - Signer: Yes.
 - Init: No.
 - PDA: No.
 - Mutable: Yes.
- **program_account:** Optional program account.
 - Signer: No.
 - Init: No.
 - PDA: No.
 - Mutable: No.
- **program_data_account:** Optional program data account.
 - Signer: No.
 - Init: No.
 - PDA: No.
 - Mutable: No.
- **system_program:** Optional system program.
 - Signer: No.
 - Init: No.
 - PDA: No.

- Mutable: No.

Additional checks and behavior

- correct amount of accounts must be passed
- `buffer` can be a PDA, in which case it must have the correct derivation + seed; otherwise must be a signer.
- `buffer` must be rent exempt

CPI

- Calls the system program's `Allocate` and `Assign` instructions to initialize accounts

5.9. Instruction: `extend`

This instruction will extend a buffer or metadata accounts size.

Input parameters

```
struct Extend {
    length: u16,
}
```

Accounts

- **account:** Buffer or metadata account to extend.
 - Signer: No.
 - Init: No.
 - PDA: No.
 - Mutable: Yes.
- **authority:** Signer of the transaction and authority for the account.
 - Signer: Yes.
 - Init: No.
 - PDA: No.
 - Mutable: Yes.
- **program_account:** Optional program account.
 - Signer: No.

- Init: No.
- PDA: No.
- Mutable: No.
- **program_data_account**: Optional program data account.
 - Signer: No.
 - Init: No.
 - PDA: No.
 - Mutable: No.

Additional checks and behavior

- correct amount of accounts must be passed
- Program ownership and writable checks are done implicitly by the runtime
- account:
 - authority must be a signer
 - must be a buffer or metadata account
 - must have a valid authority
 - must be rent exempt

6. Assessment Results

During our assessment on the scoped Program Metadata programs, we discovered two findings, both of which were low impact.

6.1. Disclaimer

This assessment does not provide any warranties about finding all possible issues within its scope; in other words, the evaluation results do not guarantee the absence of any subsequent issues. Zellic, of course, also cannot make guarantees about any code added to the project after the version reviewed during our assessment. Furthermore, because a single assessment can never be considered comprehensive, we always recommend multiple independent assessments paired with a bug bounty program.

For each finding, Zellic provides a recommended solution. All code samples in these recommendations are intended to convey how an issue may be resolved (i.e., the idea), but they may not be tested or functional code. These recommendations are not exhaustive, and we encourage our partners to consider them as a starting point for further discussion. We are happy to provide additional guidance and advice as needed.

Where Zellic states that a finding has been addressed or fixed in one or more specific commits, this indicates only that those commits introduce changes that, in our assessment, address the finding relative to the codebase version originally reviewed. This does not imply that Zellic reviewed other changes contained in those commits, the overall state of the codebase at those commits, or the interplay between remediation measures for different findings.

Finally, the contents of this assessment report are for informational purposes only; do not construe any information in this report as legal, tax, investment, or financial advice. Nothing contained in this report constitutes a solicitation or endorsement of a project by Zellic.